WINMOL stem segmentation — method corrections and scale-augmentation results

Christian Winkelmann

14 August 2026

Contents

1 Executive summary

Two independent bodies of work, in the order they should be read.

1.1 Part I — Corrections to the published WINMOL method

Read directly from `WINMOL_segmentor` (R) and `WINMOL_Analyzer` (Python), with every value quoted from source or measured by running it. Five items change how the published results should be read, and three of them are places where the papers and the code disagree.

#	Finding	Consequence
1	**The composite F1/BCE loss does not train.** `F1Score_loss` calls the *metric*, which applies `k_round`; a rounded value has zero derivative almost everywhere.	The model trains on **plain BCE**. The term appears in the reported loss and is absent from the optimisation. Verified three ways, including an 8-epoch R run agreeing with pure BCE to **eight decimal places**.
2	**The split is a random 80/20 over tiles**, drawn per stage — not site-level, not spatial. With 100× oversampling, tiles overlap.	Early stopping, the LR schedule and checkpoint selection all read an optimistic validation signal.
3	**Tile side length is 15 m, not ≈15.4 m**, fixing effective GSD at **2.93 cm/px**.	Load-bearing: matching training GSD to it was worth **+14.6 F1** in our measurements.
4	**Node spacing is 50 cm, not the documented 25 cm** (`min(min_length, 0.5)`).	Measured 0.47–0.50 m on real output across four orthomosaics.
5	**A transposed digit in the decoder** (265 filters where the symmetric ladder gives 256).	The published `.hdf5` is 31,140,642 parameters; the described architecture gives 31,036,673. Not reproducible from the paper without the typo.

Plus two coordinate traps in the vectoriser that produce plausible geometry rather than an error — the skeletonisation pad is never removed, and the EDT diameter branch reads pixel `(row, col)` as easting/northing. Full detail with file and line references in Part II.

A reproduction worth noting. Trained single-stage on the published SpecDS and scored on the published TestDS with our own metric code: **0.7581**, which matches the 2022 paper's *best two-stage pretrained* result (75.6%) without any GenDS pretraining. The published beech model scores **0.6812** on that same TestDS — below the paper's figure, unexplained, and worth raising with the author.

1.2 Part II — Scale augmentation

The Analyzer's effective resolution is `tile_size / 512`, a user-set knob most people never touch deliberately. Jittering the training footprint ±30% makes a model robust to it:

	fixed spread	jitter spread	improvement	cost at 1.00×
HRNet, within-site	0.0295	0.0207	−0.0087 (t −6.0)	−0.0011 (noise)
UNet, within-site	0.0334	0.0146	−0.0188 (t −57.0)	**+0.0020** (3/3 seeds)
UNet, clean site holdout	0.1231	0.1116	−0.0115	**+0.0328** (3/3 seeds)

Recommendation: enable it by default. It costs nothing at the deployment scale, removes 30–56% of the degradation when the tile size moves, and on the one clean site holdout where segmentation works at all it is worth **+3.3 F1** at the deployment scale.

It does **not** replace matching the training scale to the serving scale — that is worth 14.6 F1 against this 0.9–3.3. Do both.

1.3 What is not settled

- **Bachsee_north returns F1 < 0.13 for every model ever trained here.** Long recorded as a colour-domain failure. It is better described as an **occlusion** failure: the canopy is closed and the stems lie beneath it, so stem-vs-background contrast is **+0.12** against **+0.80** at Kaufland. Its labels were suspected of misregistration — annotations in EPSG:25833, orthomosaic in EPSG:32633 — but measured on the orthomosaic's own grid **every site peaks at zero offset**, so the labels are correctly placed and the datum mismatch is harmless. The site is a genuine limit of the imagery, not a data defect.
- **The site holdout rests on one informative fold.** Of four beech sites, two are the same forest 4.4 years apart (33% footprint overlap) and one is Bachsee_north.
- **Label scarcity.** Under 1 ha of stem is labelled in total; beech is near exhausted and pine has none.

2 Reder's method: gaps closed from source, and where the docs disagree with the code

The methodological report on Reder et al. 2022 / 2025 lists items "NOT recoverable from openly accessible sources" and points at `WINMOL_segmentor` (R) and `WINMOL_Analyzer` (Python) as the authoritative places to read them. Both were read directly. This closes what is closeable and flags three places where the published description and the code disagree.

Every value below is quoted from source or measured by running it, with the file and line.

2.1 1. Hyperparameters — closed

item	value	source
optimizer	Adam, `lr = 1e-3`, β_1 0.9, β_2 0.999, `decay = 0`, `amsgrad = FALSE`	`controlling.R`
batch size	**4**	`main_training.R`
epochs	100 cap per stage	`training.R`
early stopping	`val_loss`, patience **3** (stage 1) / **5** (stage 2)	`callbacks.R:15–17`
LR schedule	ReduceLROnPlateau, factor 0.1, patience 2, `min_delta = 1e-4`	`callbacks.R:7–12`
checkpoint	`save_best_only = TRUE` on `val_loss`	`callbacks.R:6`
augmentation	flips + brightness/contrast/saturation/hue, train split only	`input_pipeline.R`

2.2 2. Split construction — closed, and it is not site-level

```r
train_data <- initial_split(train_data, prop = 0.8)   # training.R, both stages
```

A **random 80/20 split over tiles**, drawn independently per stage. Not spatial, not site-level. The report infers a site-aware evaluation from the 15-site design; the training code does not implement one. With the generator's 100× oversampling, tiles overlap, so this validation signal is optimistic — which matters because early stopping, the LR schedule and checkpoint selection all read it.

2.3 3. The composite loss does not train — measured

Both papers present the F1/BCE composite as the key choice for foreground/background imbalance. In code:

```r
F1Score <- custom_metric("F1Score", function(y_true, y_pred) {
  y_pred <- k_round(y_pred)          # <- hard threshold
  ... })
F1Score_loss <- function(y_true, y_pred) {
  loss_binary_crossentropy(y_true, y_pred) + (1 - F1Score(y_true, y_pred)) }
```

`F1Score_loss` calls the **metric**, and the metric rounds. `k_round` has zero derivative almost everywhere, so the F1 term contributes no gradient: **the model trains on plain binary cross-entropy.** The term is real in the reported loss value and absent from the optimisation.

Verified three independent ways:

- **In torch** — R's expression transcribed gives a gradient `torch.equal` to plain BCE's (grad-norm 0.012018 for both), while the reported value is ~2× larger.
- **In R itself** — `LOSS=f1` (Stefan's untouched `controlling.R`) against `loss_binary_crossentropy` alone, same seed, 8 epochs: precision, recall and F1 agree to **eight decimal places** at every epoch. Only `loss` differs, by exactly `1 − F1`.
- **Ablated** — replacing it with a *differentiable* soft-F1 term, paired over seeds, shifts recall **+3.8** and precision **–2.5** (4/4 seeds, paired t 4.86 / –5.47). So the term does something once it has a gradient; in the published form it does nothing.

This does not invalidate the results — BCE is a reasonable loss — but the stated rationale for the loss choice is not what the model experienced.

2.4 4. Architecture — a transposed digit

`model_UNet.R:87,91`, decoder stage E7:

```
layer_conv_2d(filters = 265, ...)    # every other stage is symmetric: 512, 256, 128, 64
```

The transpose conv above it correctly emits 256. Both convs of that block use **265**. The published `.hdf5` loads at **31,140,642** parameters; the symmetric ladder gives 31,036,673. Harmless numerically, but "31M U-Net" is not reproducible from the paper's description without the typo.

2.5 5. Instance-extraction defaults — closed

`classes/Config.py`:

parameter	default	GUI label in the report
`min_length`	**2.0** m	Min Length
`max_distance`	**8**	Max Distance
`tolerance_angle`	**7**	Maximum Angle
`max_tree_height`	**32** m	Maximum Tree Height
`tile_size`	**15** m	Tile side length
`img_width`	**512** px	Tile pixel extent
`overlap_pred`	8	—
`stem_binary_threshold`	0.5	—

Tile side length is 15 m, not ≈15.4 m. The report derives 15.36 m from 3 cm × 512 px; the code fixes 15 m, giving an effective GSD of **15/512 = 2.93 cm/px**. That number is load-bearing: matching training GSD to it was worth **+14.6 F1** in our own measurements.

Skeletonisation (`utils/Skeletonization.py`): `skimage.morphology.skeletonize` on the binary mask, after padding by `int(max_tree_height / px_size) + 1`. Endpoints and branchpoints are found by crossing-number (transitions `==` 1 endpoint, ≥ 3 branchpoint); branchpoints are removed, dense nodes eroded, then segments refined to the measuring-point spacing with a minimum length of `floor((min_length/4) / px_size)`.

Diameter: two methods, `diameter_method = "contour"` (default) or `"edt"`, measured on a local normal (`_local_normal`, `_measurement_vector`, `diameter_vector_half_length_m = 1.0`) — so "perpendicular to the centerline" is confirmed.

Volume: `calc_l_v(p1, p2, d1, d2)`, truncated cone between consecutive nodes. Confirmed.

2.6 6. Node spacing is 50 cm, not 25 cm

The WINMOL documentation says the diameter is determined "every 25 cm". The code says:

```
measuring_point_spacing_m = 0.5                                  # Config.py:104
measuring_point_spacing = math.floor(
    min(config.min_length, config.measuring_point_spacing_m) / px_size)   # Skeletonization.py
```

`min(2.0, 0.5) = 0.5` -> **50 cm**. Measured on real output across four orthomosaics, the mean gap between consecutive centreline vertices is **0.47–0.50 m**. Either the docs describe a different version or the figure is wrong; the shipped default is 50 cm.

2.7 7. Two coordinate traps in the vectoriser

Both produce plausible geometry rather than an error, so they are invisible without checking:

- **The skeletonisation pad is never removed.** `find_segments` pads by `max_tree_height / px_size` and returns coordinates still carrying that offset — 1530 px at 2.09 cm/px. The comment at `Skeletonization.py:175` claims the offset is removed; `np.argwhere` returns raw padded indices.
- **Paths are pixel `(row, col)`, not world coordinates**, and `path.length` is a pixel count. World coordinates are applied only at write time. `calc_v_d_edt` passes `stem.path.coords` to `_xy_to_rowcol`, which reads them as easting/northing — so the **EDT diameter branch appears broken**; the default contour path is self-consistent in pixel space.

2.8 8. CLI: `Stems` does not vectorise

`run_stem_pipeline` calls the prediction phase and stops; only `run_tree_pipeline` (`Trees`) runs `run_vector_phase`. A `Stems` run exits cleanly with a stem raster and no vectors — which reads as a failure and is not one.

2.9 What this changes in the report's recommendations

Stage 1 (data). Target the code's actual scale — 15 m / 512 px = 2.93 cm/px — rather than "3 cm". Our measurement: serving a model at a GSD it was not trained on cost 14.6 F1, and the same published beech model scores **0.8003** on native-512 tiles versus **0.6812** on Reder's own 313 px TestDS (upsampled ×1.64 to reach the input). Resolution handling, not model quality, explains a 12-point swing.

Stage 2 (splits). The report's advice to use site-level holdout is right and the code does not do it. Worth knowing what it costs: in our corpus a whole-site holdout costs 8–11 F1, and one site (November beech canopy, 100% amber pixels) returns F1 **exactly 0.0000** — so a leave-one-site-out mean can be carried entirely by one dead fold. Report per-fold.

Stage 2 (copy-paste + two-stage). GenDS tiles are 2000×2000 but carry ~500–650 px of real information — round-trip loss through 512 is 1.45 grey levels at 2× against SpecDS's 3.73, and edge energy is 7.97/px against 15.52. Effective GSD ≈ 4.6 cm/px, which matches SpecDS's 4.8 cm/px, so the two stages are well aligned with each other but not with the Analyzer default. On our own corpus, synthetic pretraining measured **–0.2 to –1.4** once a validation-split bug was fixed (it had read +1.1 to +3.6 before).

Stage 4 (evaluation). Pixel F1 is not comparable to DR25 — different quantity. Also fix a noise floor before comparing anything: two runs with **bit-identical gradients** landed 2.0 F1 apart without deterministic kernels, 0.5–1.0 with. Single-run differences below that are unreadable, which is why the ablations above are paired over 5 seeds.

2.10 A reproduction worth noting

The 2022 paper reports pixel F1 **72.6%** for SpecDS-only training and **75.6%** for its best two-stage pretrained model, on TestDS.

Trained single-stage on the same published SpecDS, scored on the same published TestDS with our own metric code: **0.7581**. That matches the paper's *best pretrained* result without any GenDS pretraining. The R U-Net retrained under the same conditions gives 0.7424.

Two caveats. The published **beech** model (`SpecDS_Beech_512`, 2023-02-28) scores only **0.6812** on that same TestDS — below the 2022 paper's figure, unexplained, and worth raising with the author. And TestDS as distributed on Zenodo contains **117** tiles, not the 106 stems the paper describes (verified by md5 against the record).

Sources read directly: `WINMOL_segmentor` — `controlling.R`, `training.R`, `callbacks.R`, `main_training.R`, `model_UNet.R`, `input_pipeline.R`. `WINMOL_Analyzer` — `Config.py`, `Skeletonization.py`, `Vectorization.py`, `Quantification.py`, `winmol_run.py`. Datasets verified byte-identical against Zenodo record 5682580.

3 Scale augmentation: ±30% footprint jitter buys robustness for free

Design and yardstick were fixed before training — `superpowers/specs/2026-08-13-scale-augmentation-design.md`. Numbers below are copied verbatim from `assets/scale-sweep.json`, rendered by `scripts/plot_scale_sweep.py` from the six per-model sweeps in `assets/scale-aug/`.

Answer: ship it. Measured on two architectures. On HRNet, jitter flattens the scale curve by **0.87 F1** (3/3 seeds, paired t −6.02) and costs **0.11 F1** at the native scale — 1/3 seeds positive, i.e. noise. On UNet the same manipulation flattens it by **1.88 F1** (3/3 seeds, t −56.95) and *gains* 0.20 F1 at native scale as well. It wins at every scale on every UNet seed.

Sections below cover HRNet first, then the UNet replication and a reproducibility check.

3.1 Setup

data	`BeechScale666` — 3388 / 400 / 400 tiles at 666 px, 19.512 m, **2.9297 cm/px**
sites	Campus + Oberheide block-split (60 m); Bachsee_north, Kaufland whole-to-train
leak check	closest train<->val<->test approach **28.3 m** / **27.9 m** vs a **27.59 m** floor
arms	crop 512–512 (*fixed*) vs 394–666 (*jitter*), `RandomSizedCrop`->512
held fixed	same tiles, rotation/flip/photometric, arch, batch 16, `--deterministic`
n	3 paired seeds per arch (HRNet, UNet); test = 400 tiles, CPU EP, threshold 0.5

Both arms train on **the same 666 px source tiles** and both get random crop *position* — only the scale distribution differs. That is what makes the comparison clean.

3.2 HRNet: per-scale result

Differences are jitter − fixed, paired within seed.

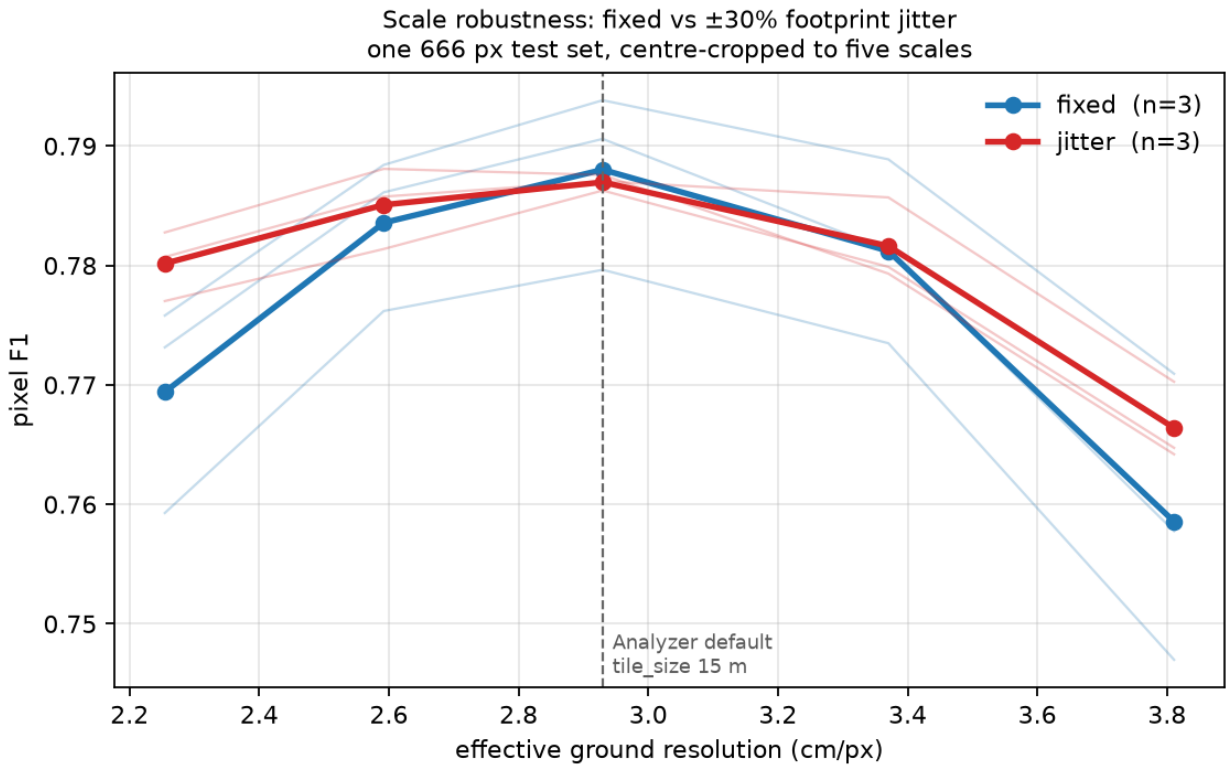


Figure 1: F1 versus effective ground resolution

crop	ratio	GSD	fixed	jitter	diff	sign	t
394	0.77×	2.254	0.7694	0.7802	**+0.0108**	3/3	2.87
453	0.88×	2.592	0.7836	0.7851	+0.0015	2/3	0.66
512	**1.00×**	**2.930**	0.7880	0.7870	−0.0011	1/3	−0.26
589	1.15×	3.370	0.7812	0.7816	+0.0004	1/3	0.14
666	1.30×	3.811	0.7586	0.7664	**+0.0078**	2/3	1.46

Drop from each arm's own peak — the robustness claim, independent of absolute level:

arm	s1	s2	s3	mean
fixed	0.0229	0.0326	0.0328	**0.0295**
jitter	0.0168	0.0215	0.0239	**0.0207**

Paired: **−0.0087, 3/3 seeds flatter, t −6.02.**

3.3 Reading it

The win is at the flanks and it is a recall win. At 1.30× the fixed arm's precision *rises* to 0.819–0.829 while recall falls to 0.680–0.728: coarser pixels make stems thinner, the model stops committing, and it under-segments. This is the benign failure mode — an under-reporting model, invisible in F1 alone. Jitter holds recall 3–6 points higher there at some precision cost. Every domain-shift failure measured in this project has had this same shape.

The per-scale t-values are weak; the spread test is not. With n=3, only the 0.77× point approaches significance on its own. But the spread statistic uses each seed's whole curve rather than one point of it, and it is unanimous with $|t| = 6.0$. The per-scale rows are best read as *where* the effect lives, not as five independent tests.

The cost at native scale is nil. −0.0011 F1, 1/3 seeds positive. The worry going in — that spreading training over 2.25–3.81 cm/px would blunt the model at the one resolution it is actually served at — did not materialise. Note also that validation ran at 1.00× (`--eval-tiling`), which structurally favours the fixed arm, so this is if anything generous to the baseline.

3.4 Replication on UNet — the effect is real and larger

Repeated identically with `--arch unet` (same dataset, crop ranges, seeds, everything); sweeps in `assets/scale-aug-unet/`, aggregate in `assets/scale-sweep-unet.json`.

crop	ratio	GSD	fixed	jitter	diff	sign	t
394	0.77×	2.254	0.7492	0.7689	**+0.0197**	3/3	14.97
453	0.88×	2.592	0.7760	0.7802	+0.0042	3/3	3.67
512	**1.00×**	**2.930**	0.7812	0.7833	+0.0020	3/3	1.82
589	1.15×	3.370	0.7729	0.7792	+0.0063	3/3	3.97
666	1.30×	3.811	0.7498	0.7690	**+0.0191**	3/3	6.13

Spread: fixed **0.0334** (0.0344/0.0333/0.0326) vs jitter **0.0146** (0.0159/0.0138/0.0142). Paired: **−0.0188, 3/3 seeds flatter, t −56.95.**

Jitter wins at **every** scale on **every** seed, and unlike HRNet it does not even cost anything at 1.00× — it gains 0.20 F1 there, 3/3 seeds.

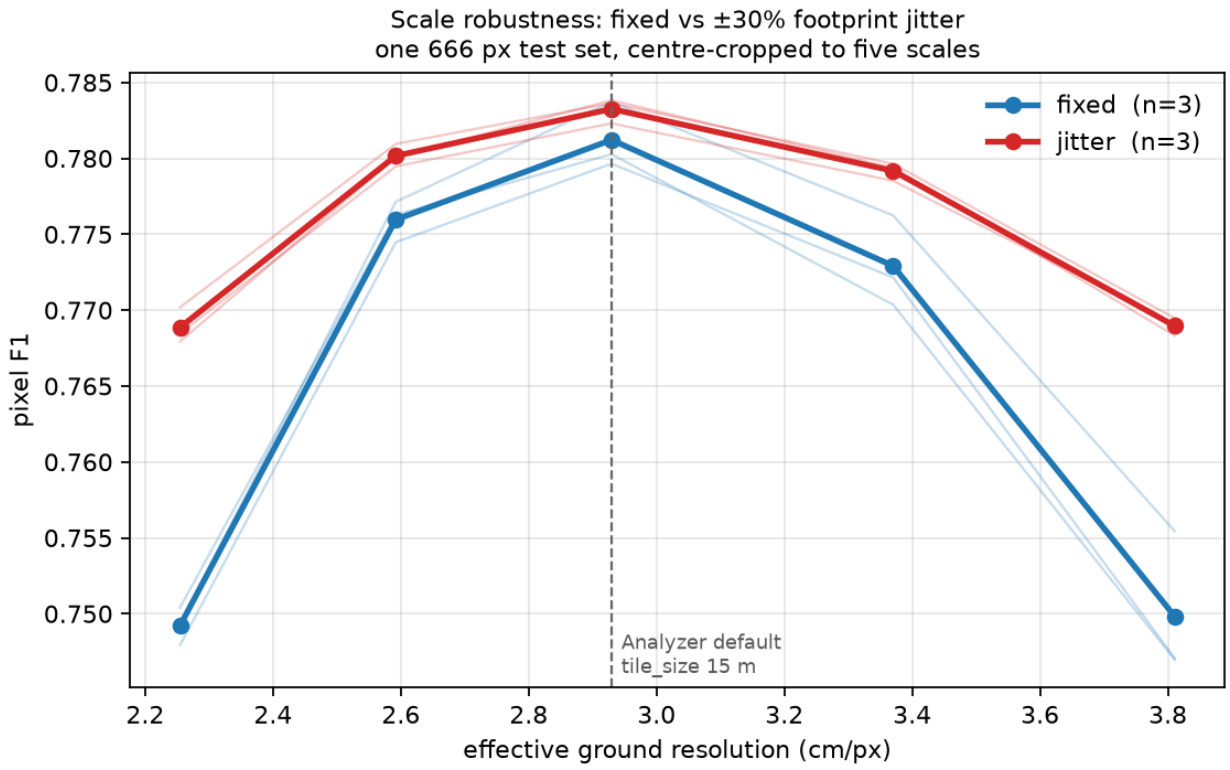


Figure 2: UNet: F1 versus effective ground resolution

Why UNet gains more. HRNet carries parallel branches at several resolutions through the whole network and fuses them repeatedly, so multi-scale context is architectural. A plain UNet has one resolution ladder and must *learn* scale invariance from the data. That predicts exactly what is measured: UNet is the more scale-brittle baseline (spread 0.0334 vs HRNet's 0.0295) and gains the most from jitter (−0.0188 vs −0.0087). The two results agree in direction and differ in magnitude in the direction the architectures predict, which is a stronger joint result than either alone.

	fixed spread	jitter spread	gain	cost at 1.00×
HRNet	0.0295	0.0207	−0.0087 (t −6.0)	−0.0011 (1/3, noise)
UNet	0.0334	0.0146	−0.0188 (t −57.0)	**+0.0020** (3/3)

3.5 Reproducibility check

`fixed-s1` scored highest in its arm in both architectures, so it was re-run from scratch with the same seed on a different GPU:

	F1	precision	recall
original (GPU 0)	0.7922	0.7990	0.7856
reproduction (GPU 7)	0.7922	0.7990	0.7856

Those are `test_results.md` figures — 1600 windows from `--eval-tiling`, not the sweep's 400 centre crops, so they differ slightly from the 1.00× column above (0.7922 vs 0.7938 for the same model). Every arm-vs-arm number in this report comes from the sweep; the table here is a run-identity check, not a scale measurement.

Identical to four decimals. `--deterministic` reproduces across devices, so between-seed differences of 1–2 F1 in the fixed arm are genuine seed variance, not run-to-run noise — and being common to both arms, they cancel in the paired differences. Runs now write `run_config.json` beside the model so this is auditable from the artifacts rather than requiring a re-run.

3.6 Caveats, stated

- **F1 is not comparable across scales.** A 23 cm stem is 10.2 px at 2.25 cm/px and 6.0 px at 3.81. The downward slope on the right of the figure is partly the task getting harder, not the models getting worse. Only the arm-vs-arm gap within a column is readable.
- **Arm B occasionally sees the full 666 px tile; arm A never does.** A wider crop range means a wider field of view. This is inherent to the manipulation and cannot be separated from scale jitter without also changing the footprint.
- **This is not the earlier "5.2 F1 across ±30% zoom" figure.** That came from a different model and a test built by re-sampling the orthomosaic per scale, which put each scale on different ground. The 2.95 F1 (HRNet) and 3.34 F1 (UNet) measured here are the clean version of the same quantity and supersede it; the two are not directly comparable.
- **Sites are shared across splits.** The claim is scale robustness within known forests, not transfer to a new site.
- **n = 3.** Enough for the unanimous spread result, thin for the per-scale rows.

3.7 Recommendation

Enable `--multiscale --crop-min-px 394 --crop-max-px 666` by default for beech training, on both architectures. It removes **30%** of the scale degradation on HRNet and **56%** on UNet, and costs nothing at `tile_size = 15` — on UNet it gains there too. Users do move the tile-size spinbox, and nothing in the model warns them when they have.

This does **not** replace matching the training scale to the serving scale. That was worth 14.6 F1; this is worth 0.9–1.9. Be robust *and* correctly scaled.

Two questions worth answering next, in order:

1. **Does a wider range keep paying?** The corpus spans 1.58–6.39 cm/px, a 4× range, against the 1.7× tested here. The UNet curve is still falling at both ends of the sweep, so ±30% is unlikely to be where the gain stops.
2. **Does it survive a site holdout?** ~~These splits share sites.~~ **Answered** — see `scale-augmentation-loso.md`. Flatter in 4/4 folds, and worth **+3.3 F1 at the deployment scale** on the one clean holdout where the model segments at all. Two of the four folds turned out not to be site holdouts (Campus and Campus_Oberheide are the same forest 4.4 years apart, 33% footprint overlap) and one was dead (Bachsee_north, F1 < 0.13 for every arm), so the new-site evidence is one strong fold rather than four.

4 Scale jitter under a site holdout — four folds, reported individually

Follow-up to `scale-augmentation-results.md`, which measured scale jitter on splits that **share sites**. This asks whether the robustness survives when the test site is absent from training. UNet, 3 paired seeds per arm per fold, 24 runs. Numbers copied verbatim from `assets/loso-*.json`.

Answer: it survives, but the evidence is thinner than the within-site result and one fold is uninformative. Jitter's scale curve is flatter in **4 of 4 folds**; only one fold reaches the usual significance bar on its own.

4.1 Two of these folds are not site holdouts

`tiles.jsonl` says **33% of Campus tiles overlap a Campus_Oberheide footprint** (centroids 183 m apart). They are the same forest imaged 2017-10-16 and 2022-02-26, and windthrown stems do not move between acquisitions — so those are largely the same physical stems.

fold	test tiles	nearest training site	what it measures
Bachsee_north	800	**5.68 km**	new-site transfer
Kaufland	388	**3.56 km**	new-site transfer
Campus	800	0 m (33% overlap)	4.4-year acquisition gap
Campus_Oberheide	800	0 m (30% overlap)	4.4-year acquisition gap

Reported under those two headings, never pooled.

4.2 Every fold, no mean

`spread` = each seed's peak-minus-worst F1 across 0.77× / 1.00× / 1.30×. Lower is more scale-robust; the paired column counts seeds where jitter is flatter.

fold	F1@1.00× fixed	jitter	spread fixed	spread jitter	flatter	t
Bachsee_north	0.0945	0.0434	0.0940	0.0778	1/3	−0.15
Kaufland	0.6737	**0.7065**	0.1231	0.1116	2/3	−1.31
Campus	0.5263	0.5290	0.1125	0.0886	2/3	−1.41
Campus_Oberheide	0.6793	0.6792	0.0408	**0.0272**	3/3	−5.52

Direction is unanimous — all four mean spreads favour jitter — but only Campus_Oberheide clears |t| ≥ 3 with unanimous seeds. State this as a fold count, not a mean: *flatter in 4/4 folds, individually significant in 1/4.*

4.2.1 Bachsee_north is a dead fold — and not for the reason recorded so far

F1 **0.03–0.13** for every arm and seed. `docs/process.md` attributes this to colour domain (amber November canopy). That is real — Bachsee's saturation is **0.73 against 0.19–0.32** everywhere else — but it is not the main cause. Look at the tiles:

Bachsee's canopy is **closed**. The stems are underneath it, glimpsed through gaps rather than seen. Measured as stem-minus-background luminance in units of each tile's own spread (`scripts/site_gallery.py`, 120 random tiles per site):

site	saturation	**stem contrast**	fold F1 @ 1.00×
Kaufland	0.32	**+0.89**	0.67 – 0.71
Campus_Oberheide	0.25	+0.43	0.68
Campus	0.19	+0.24	0.53
Bachsee_north	0.73	**–0.17**	**0.04 – 0.09**

Fold F1 is monotone in stem contrast. Bachsee is the only site where the sign flips: its labelled stems are *darker* than their surroundings and separated by a sixth of a standard deviation. Kaufland's are brighter by nearly a whole one.

That changes the interpretation. This is not a model that fails on amber imagery — it is imagery in which the target is largely **not visible**, so no model can recover the labels from it. The predictions are near-empty, which is the recall collapse in its extreme form:

Green is ground truth, red is prediction; on most tiles there is no red at all. So the fold is a **tie**, not evidence for or against jitter, and its per-scale differences are ratios of noise. Had these four folds been averaged, this one would have moved the mean while carrying no information.

4.2.1.1 The labels are not misplaced — checked

If the stems are barely visible, the obvious suspicion is that the polygons came from elsewhere and are misregistered. Bachsee_north invites it: its annotations are **EPSG:25833** and its orthomosaic **EPSG:32633**, the same UTM zone on different datums, a pair that has diverged ~0.5 m through plate motion. `docs/training-data-from-annotations.md` records the mismatch as a known defect.

Tested with `scripts/check_registration.py`, which rasterises the polygons onto the orthomosaic's **own unrotated grid** and slides them over a grid of world offsets, looking for where stem pixels are brightest against their surroundings:

site	stems / ortho CRS	contrast at zero	bright peak	verdict
Bachsee_north	25833 / 32633 **mismatch**	**+0.118**	(0.00, 0.00) m	registered
Campus	32633 / 32633	+0.384	(0.00, 0.00) m	registered
Campus_Oberheide	25833 / 25833	+0.811	(0.00, 0.00) m	registered
Kaufland	25833 / 25833	+0.797	(0.00, 0.02) m	registered

Every site peaks at zero offset. The CRS mismatch is real but harmless — the pipeline's `transform_geom` applies the datum shift, leaving a residual under 4 cm. So the labels are correctly drawn *and* correctly placed; Bachsee's stems are simply 7× fainter than Kaufland's (+0.118 against +0.797 at zero shift, native resolution).

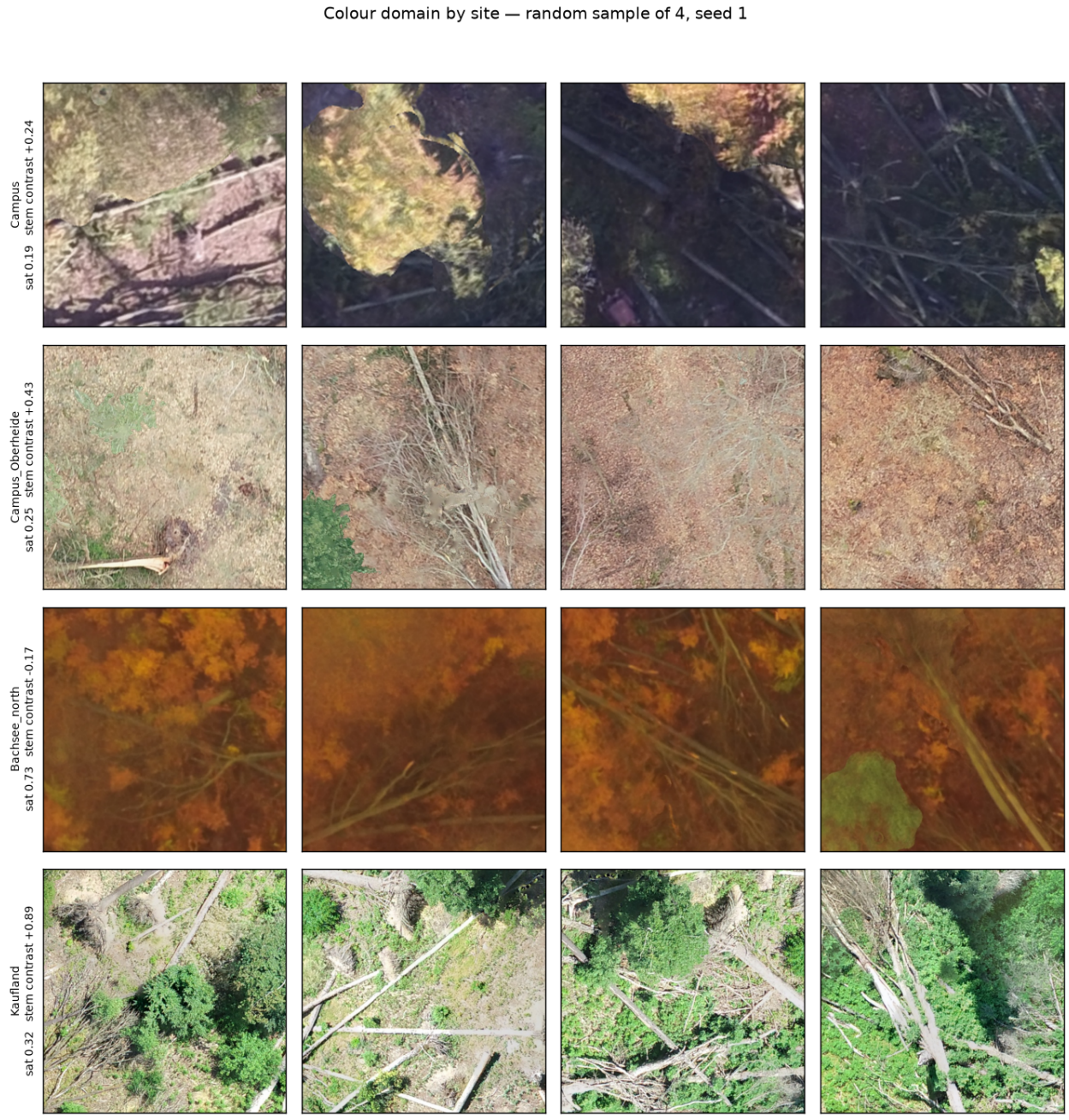


Figure 3: Colour domain by site

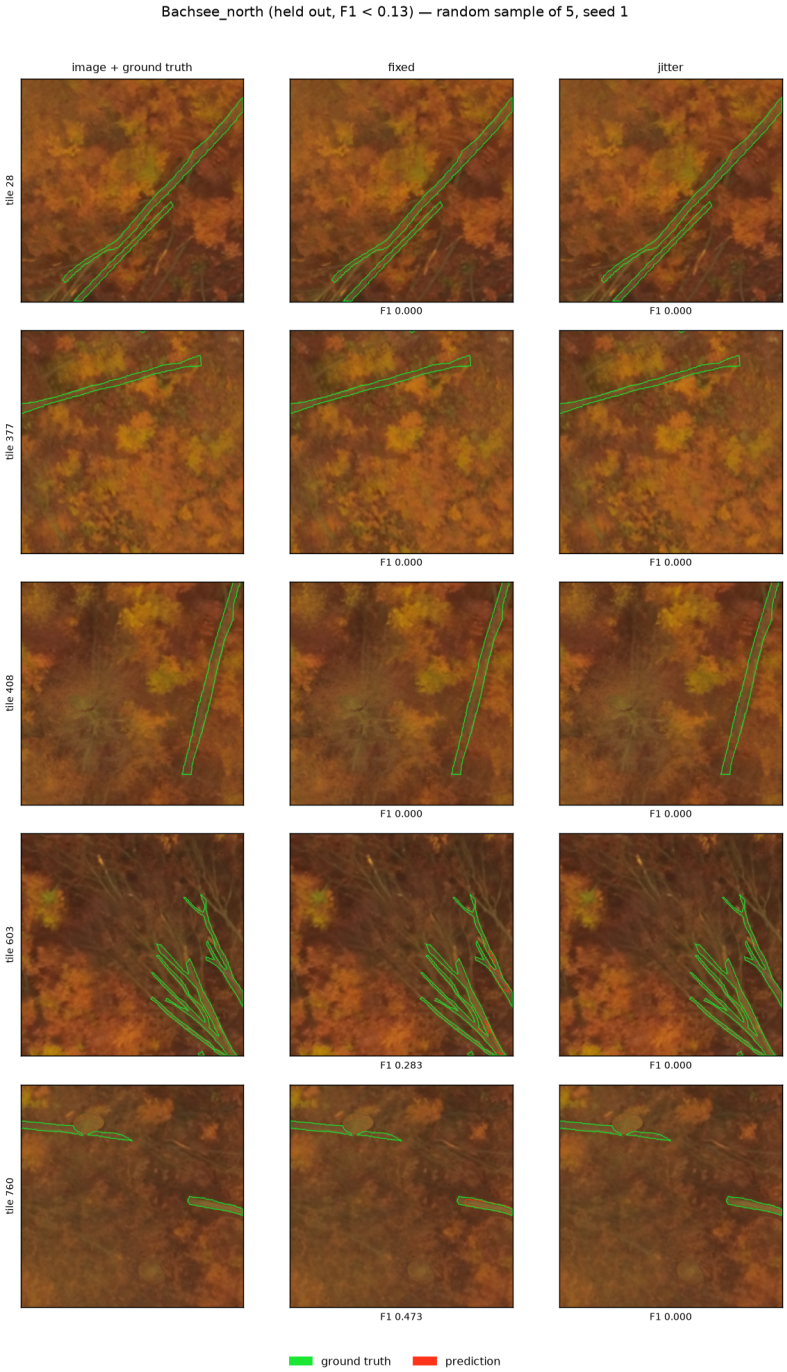


Figure 4: Bachsee_north examples

Two traps this measurement had to avoid, both of which produced a confident wrong answer first:

- **Measuring in tile-pixel space.** Training tiles are cut at random rotations, so a constant world offset lands at a different pixel offset in every tile and averaging smears it away. A first pass this way appeared to show a 0.53 m shift that did not exist.
- **Ranking by |contrast|.** A stem's shadow lies a stem-width away and gives strong *negative* contrast, so the largest absolute response is usually the shadow. That reported Campus — which peaks at zero — as offset by 0.41 m.

4.2.2 Kaufland is the informative clean holdout

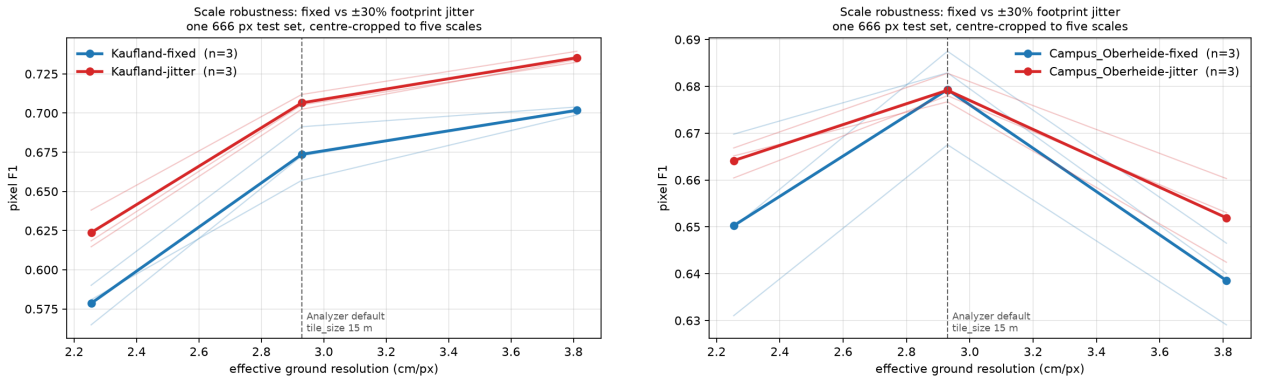
The only true spatial holdout where the model works at all, and jitter wins **at every scale on every seed**:

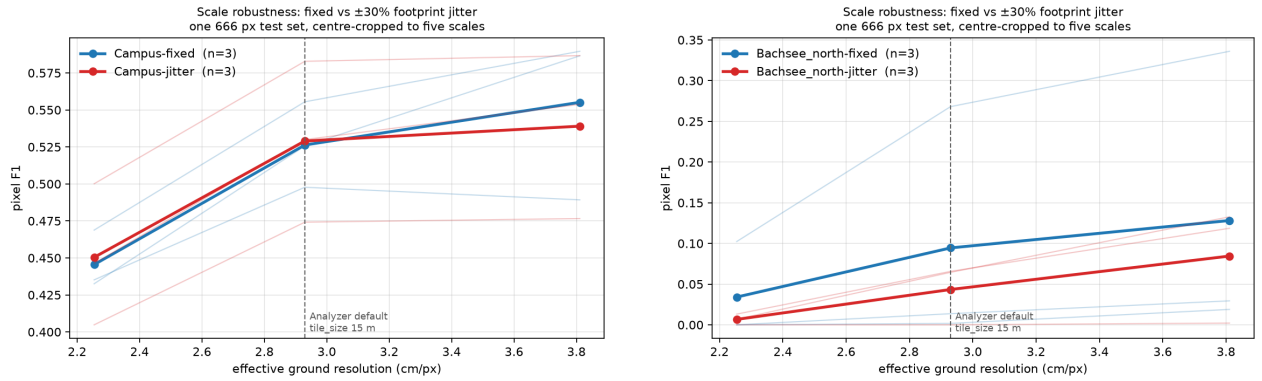
ratio	GSD	fixed	jitter	diff	sign	t
0.77×	2.254	0.5786	0.6237	**+0.0451**	3/3	4.37
1.00×	2.930	0.6737	0.7065	**+0.0328**	3/3	2.61
1.30×	3.811	0.7017	0.7353	**+0.0336**	3/3	9.58

Here jitter is worth **+3.3 F1 at the deployment scale** — an order of magnitude more than the +0.2 measured within-site. Note the curve *rises* with coarseness on this fold, unlike every other: Kaufland is the corpus's finest orthomosaic (2.09 cm/px native) and its densest (stem fraction 0.063 against 0.028–0.038), so its peak sits outside the sampled range. Its spread is therefore a lower bound.

4.3 The curves

Per fold, F1 against effective ground resolution, per-seed spread drawn rather than summarised. Note the y-scales differ by an order of magnitude between folds.





4.4 What the holdout costs

	within-site (pooled blocks)	site holdout
F1 @ 1.00×	0.78	0.04 – 0.71

Between 7 and 74 F1 points, depending entirely on which site. Any single-fold number here would have been arbitrary — which is the argument for reporting all four.

4.5 Caveats

- **n = 3 seeds per arm per fold.** Under domain shift the within-arm seed spread reaches **12 F1 points** (Campus fold), against 1–2 within-site. Absolute F1 differences below ~3 points are unreadable at this n; the spread statistic is within-model and survives better, which is why the claim rests on it.
- **Only 2 of 4 folds are true spatial holdouts**, and one of those is dead. The new-site evidence is effectively **one fold** — strong, but one.
- **Train sizes differ by fold** (2788–4000 tiles), inherent to leave-one-site-out.
- **Val comes from one site** in the Campus and Campus_Oberheide folds, since only those two sites are large enough to block-split at a 19.512 m footprint.
- **Corpus limit.** Four beech sites, two of them the same ground. A stronger claim needs annotated sites this corpus does not have — see winmol-label-scarcity.

4.6 Verdict

The within-site recommendation stands and is mildly strengthened: jitter never hurt in any fold, flattened the curve in all four, and on the one clean holdout where segmentation works at all it was worth +3.3 F1 at the deployment scale. But calling this “validated across sites” would overstate a corpus with one informative holdout fold.

5 Working process

5.1 Splits

Standard: no tile footprint may overlap between train, val and test. Spatial autocorrelation inside a split is accepted and expected.

`make_splits.py --strategy blocks` satisfies this by construction: the AOI is cut into a grid, whole blocks are dealt to splits, and each block is then shrunk by the footprint half-diagonal. Two centres on opposite

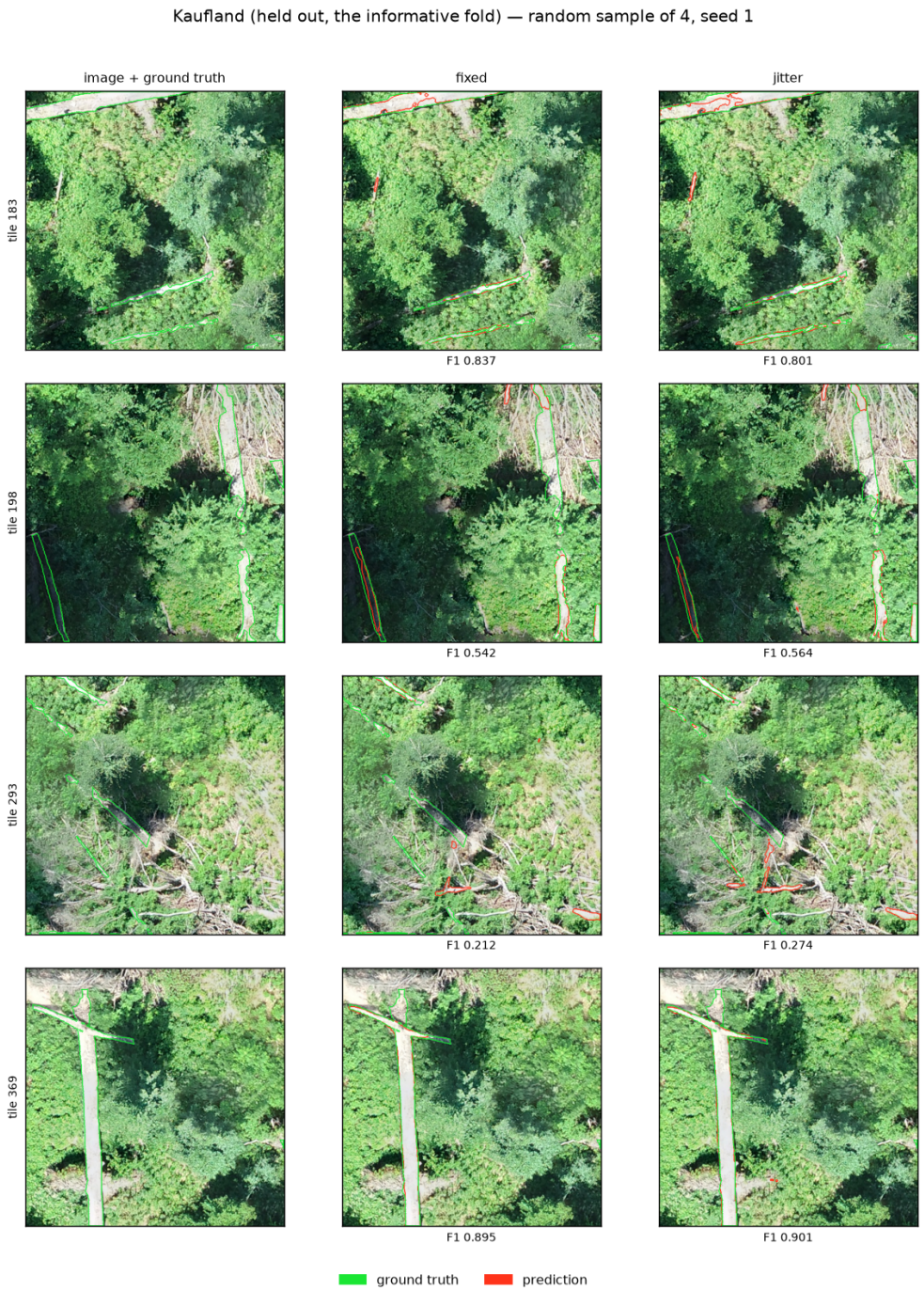


Figure 5: Kaufland examples

sides of a shared edge are therefore at least `extent · √2` apart — the distance below which two rotated footprints can share a pixel.

Verification is a pass/fail check, not a discussion. `scripts/plot_splits.py` prints the measured closest centre-to-centre distance per site and exits non-zero if a site is missing from a split. Run it after every build; report the number once.

	requirement
footprint overlap between splits	none — closest centre distance > `extent · √2`
spatial autocorrelation within a split	accepted
every site in every split	yes, unless the AOI is too small to cut (record in `splits.json`)
provenance	`tiles.jsonl` per split: site, centre, angle, GSD, stem fraction

A site whose AOI cannot yield 12+ usable blocks goes wholesale into one split via `whole_split`, recorded in `splits.json`. No further comment needed.

5.2 Scale

Train at the scale the Analyzer serves. Effective GSD is `extent_m / tile_px` in training and `tile_size / 512` in the Analyzer. These must match, or the model is served at a different resolution than it learned.

Default `tile_size = 15` -> **2.93 cm/px** -> build with `--extent 15 --tile-px 512`.

Measured on Kaufland, AOI-masked, same model family:

trained at	served at	F1
2.00 cm/px	2.93 cm/px (mismatched)	0.7389
2.00 cm/px	2.00 cm/px (matched)	0.8091
2.93 cm/px	2.93 cm/px (matched, default)	**0.8844**

Ship the `tile_size` alongside any model that is not trained at 2.93 cm/px.

Jitter the training footprint ±30% — `--multiscale --crop-min-px 394 --crop-max-px 666` on a 666 px source. Measured on two architectures, 3 paired seeds each: it flattens the F1-vs-GSD curve by 0.87 F1 on HRNet and 1.88 on UNet, and costs nothing at the native scale. See `scale-augmentation-results.md`. Matching the scale is still worth far more than being robust to it — do both.

5.3 Evaluation

- Tile-level F1 for comparing training runs.
- **Full-orthomosaic inference masked to the AOI** for anything claiming deployment performance. Outside the windthrow polygon stems are real but undigitised; scoring the whole raster is meaningless (measured: 0.2974 vs 0.8844 on the same prediction).
- Report precision and recall separately.

5.4 Reporting

- Numbers come from each run's own `test_results.md`, collected by `scripts/collect_results.py` into JSON. Reports render from that JSON.
- State the test set and the training scale beside every figure.
- One line per caveat, in a caveats section. Not inline.

5.5 Pipeline order

`fix –> sample –> split`, enforced by `make_splits.py`. Geometry repair happens first because GEOS aborts on the first set operation touching an invalid ring.

5.6 Environments

what	where
corpus	`/Volumes/storage/Datasets/Winmol/training_data/WINMOL`
derived tiles	`data/` in this repo, git-ignored, regenerable from `configs/*.json`
training	carrot `/raid/cwinkelmann/winmol` — check GPUs are free before launching
Analyzer	`~/hnee/WINMOL_Analyzer`, needs Python ≥ 3.10